1. Keep unit interfaces small (Parametroak maximo 4-ra limitatu)

1. Hasierako kodea: DataAccess.java

```
public void createOrder(String description, String title, float priceMin,
float priceMax, String email) {
        if (!db.isOpen()) {
            System.out.println("¡Error! La base de datos estaba cerrada.
Reabriendo...");
        }
          db.getTransaction().begin();
        try {
            Registered user = db.find(Registered.class, email);
            Order newOrder = new Order(description, title, priceMin,
priceMax, user);
            db.persist(newOrder);
            user.getOrders().add(newOrder);
            db.getTransaction().commit();
            System.out.println("Orden creada con éxito en la DB");
        } catch (Exception e) {
            db.getTransaction().rollback();
            e.printStackTrace();
        }
}
```

2. Errefaktorizatutako kodea:

Domain paketean sortutako OrderDTO klase berria:

```
public class OrderDTO {
        public String description;
        public String title;
        public float priceMin;
        public float priceMax;

        public OrderDTO(String description, String title, float priceMin,
float priceMax) {
            this.description = description;
            this.title = title;
            this.priceMin = priceMin;
            this.priceMax = priceMax;
        }
    }
```

DataAcces klaseko metodoa aldatuta:

```
public void createOrder(OrderDTO order, String email) {
          if (!db.isOpen()) System.out.println("¡Error! La base de datos
estaba cerrada. Reabriendo...");
          db.getTransaction().begin();

         try {
            Registered user = db.find(Registered.class, email);
            Order newOrder = new Order(order.description, order.title,
order.priceMin, order.priceMax, user);
            db.persist(newOrder);
            user.getOrders().add(newOrder);
            db.getTransaction().commit();
            System.out.println("Orden creada con éxito en la DB");
        } catch (Exception e) {
            db.getTransaction().rollback();
            e.printStackTrace();
        }
     }
```

3. Deskribapena:

CreateOrder metodoak "Keep unit interfaces small" bad smell-a zuen, 5 parametro zituelako. Hori dela eta, arazoa zuzentzeko, produktuaren datuak biltzen dituen OrderDTO objektua sortu dut, bertan produktuaren lau aldagaiak gordetzeko. Horren ondorioz, metodoaren parametro kopurua 2-ra murriztu da.

4. Egilea: Danel Larreategui

2. Duplicate code (Kode errepikatua ekidin)

1. Hasierako kodea: DataAccess.java

```
public void addFavorite(String email, Sale s) {
        db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            Sale saleDB = db.find(Sale.class, s.getSaleNumber());
            if (u != null && saleDB != null &&
!u.getFavorites().contains(saleDB)) {
                u.getFavorites().add(saleDB);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
    }

public void removeFavorite(String email, Sale s) {
        db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            Sale saleDB = db.find(Sale.class, s.getSaleNumber());
            if (u != null && saleDB != null) u.getFavorites().remove(saleDB);
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
}
```

2. Errefaktorizatutako kodea: DataAccess.java

```
private void updateFavorite(String email, Sale s, boolean isAdd) {
        db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            Sale saleDB = db.find(Sale.class, s.getSaleNumber());
            if (u != null && saleDB != null) {
                if (isAdd && !u.getFavorites().contains(saleDB)) {
                    u.getFavorites().add(saleDB);
                } else if (!isAdd) {
                    u.getFavorites().remove(saleDB);
                }
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
    }

public void addFavorite(String email, Sale s) {
  updateFavorite(email, s, true);
}
public void removeFavorite(String email, Sale s) {
  updateFavorite(email, s, false);
}
```

3. Deskribapena:

addFavorite eta removeFavorite metodoek "Duplicate code" bad smell-a zuten, datu-baseko transakzioen egitura bera errepikatzen zutelako. Hori dela eta, biak fusionatzeko metodo berri bat sortu dut. Metodo berri honek, parametro boolear bat jasotzen du, faboritoak gehitu ala kendu behar diren zehazteko. Orain, addFavorite eta removeFavorite metodoek funtzio berri horri soilik deitzen diote, kode errepikapena ekiditen.

4. Egilea: Danel Larreategui

3. Write short units of code (Maximo 15 lerro)

1. Hasierako kodea: DataAccess.java

```
public void acceptComplaint(Sale s) {
        db.getTransaction().begin();
        try {
            Sale sale = db.find(Sale.class, s.getSaleNumber());
            if (sale != null && sale.getComplaint() != null) {
                Complaint comp = sale.getComplaint();
                Registered buyer = comp.getSender();
                Registered seller = comp.getReceiver();

                float price = sale.getPrice();
                if (buyer != null) {
                     buyer.getSentComplaintList().remove(comp);
                     buyer.getPurchasedSales().remove(sale);
                    buyer.setMoneyKop(buyer.getMoneyKop() + price);
                    addMovement(buyer.getEmail(),
ResourceBundle.getBundle("Etiquetas").getString("Devolucion") + ": " +
s.getTitle() , price);
                }
                if (seller != null) {
                     seller.getReceivedComplaintList().remove(comp);
                    seller.setMoneyKop(seller.getMoneyKop() - price);

addMovement(seller.getEmail(),ResourceBundle.getBundle("Etiquetas").getString(
"Devolucion") + ": " + s.getTitle() , -price);
                }
                sale.setComplaint(null);
                sale.setAlreadyComplaint(false);
                sale.setSold(false);
                db.remove(comp);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
            e.printStackTrace();
        }
    }
```

2. Errefaktorizatutako kodea: DataAccess.java

```
public void acceptComplaint(Sale s) {
        db.getTransaction().begin();
        try {
            Sale sale = db.find(Sale.class, s.getSaleNumber());
            if (sale != null && sale.getComplaint() != null) {
                acceptComplaintLagun(sale, sale.getComplaint());
                db.remove(sale.getComplaint());
                sale.setComplaint(null);
                sale.setAlreadyComplaint(false);
                sale.setSold(false);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
            e.printStackTrace();
        }
    }

    private void acceptComplaintLagun(Sale sale, Complaint comp) {
        Registered buyer = comp.getSender();
        Registered seller = comp.getReceiver();
        float price = sale.getPrice();

        if (buyer != null) {
            buyer.getSentComplaintList().remove(comp);
            buyer.getPurchasedSales().remove(sale);
            buyer.setMoneyKop(buyer.getMoneyKop() + price);
            addMovement(buyer.getEmail(),
ResourceBundle.getBundle("Etiquetas").getString("Devolucion") + ": " +
sale.getTitle() , price);
        }
        if (seller != null) {
            seller.getReceivedComplaintList().remove(comp);
            seller.setMoneyKop(seller.getMoneyKop() - price);
addMovement(seller.getEmail(),ResourceBundle.getBundle("Etiquetas").
getString("Devolucion") + ": " + sale.getTitle() , -price);
        }
    }
```

3. Deskribapena:
acceptComplaint metodoak "Write short units of code" bad smell-a zuen, 25 lerro baino gehiago zituelako. Hori dela eta, diru-itzulketen eta erabiltzaile-zerrenden garbiketaren logika acceptComplaintLagun metodora atera dut, jatorrizko metodoa sinpleagoa eta laburragoa izan dadin.

4. Egilea: Danel Larreategui

4. Write simple units of code (Maximo 4 branch points)

1. Hasierako kodea: DataAccess.java

```
public boolean buyCart(String buyerEmail, List<Sale> cart) {
         db.getTransaction().begin();
         try {
             Registered buyer = db.find(Registered.class, buyerEmail);
             if (buyer == null) return false;

             float total = 0;
             for (Sale s : cart) total += s.getPrice();

             if (buyer.getMoneyKop() < total) {
                 db.getTransaction().rollback();
                 return false;
             }

             for (Sale saleFromGui : cart) {
                 Sale s = db.find(Sale.class, saleFromGui.getSaleNumber());
                 if (s == null || s.isSold()) {
                     db.getTransaction().rollback();
                     return false;
                 }

                 Registered seller = s.getUser();
                 float price = s.getPrice();
                 buyer.setMoneyKop(buyer.getMoneyKop() - price);
                 seller.setMoneyKop(seller.getMoneyKop() + price);
                 buyer.addPurchasedSale(s);
                 s.setSold(true);
                 if (buyer.getFavorites() != null) {
                     buyer.getFavorites().remove(s);
                 }
                 addMovement(buyerEmail,
ResourceBundle.getBundle("Etiquetas").getString("Buy") + ": " + s.getTitle(),
-price);
```

```
                addMovement(seller.getEmail(),
ResourceBundle.getBundle("Etiquetas").getString("Sell") + ": " + s.getTitle(),
price);
            }
            buyer.getCart().clear();
            db.getTransaction().commit();
            return true;
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
            e.printStackTrace();
            return false;
        }
    }
```

2. Errefaktorizatutako kodea: DataAccess.java

```
public boolean buyCart(String buyerEmail, List<Sale> cart) {
        db.getTransaction().begin();
        try {
            Registered buyer = db.find(Registered.class, buyerEmail);
            if (buyer == null || !diruNahikoaDu(buyer, cart)) {
               db.getTransaction().rollback();
               return false;
           }

            for (Sale saleFromGui : cart) {
               if (!erosketaBakarra(buyer, saleFromGui)) {
                   db.getTransaction().rollback();
                   return false;
               }
            }
            buyer.getCart().clear();
            db.getTransaction().commit();
            return true;
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
            return false;
        }
    }

    private boolean diruNahikoaDu(Registered buyer, List<Sale> cart) {
        float total = 0;
```

```
          for (Sale s : cart) total += s.getPrice();
        return buyer.getMoneyKop() >= total;
    }

    private boolean erosketaBakarra(Registered buyer, Sale saleFromGui) {
        Sale s = db.find(Sale.class, saleFromGui.getSaleNumber());
          if (s == null || s.isSold()) return false;

          Registered seller = s.getUser();
          buyer.setMoneyKop(buyer.getMoneyKop() - s.getPrice());
          seller.setMoneyKop(seller.getMoneyKop() + s.getPrice());
          buyer.addPurchasedSale(s);
          s.setSold(true);
          if (buyer.getFavorites() != null) buyer.getFavorites().remove(s);

        addMovement(buyer.getEmail(),
ResourceBundle.getBundle("Etiquetas").getString("Buy") + ": " + s.getTitle(),
-s.getPrice());
          addMovement(seller.getEmail(),
ResourceBundle.getBundle("Etiquetas").getString("Sell") + ": " + s.getTitle(),
s.getPrice());
        return true;
    }
```

3. Deskribapena:

Jatorrizko buyCart metodoak adarkatze-puntu (branch point) gehiegi zituen (4ko muga gaindituz), bere barnean hainbat for begizta, if baldintza eta return itzulera zituelako. Arazoa konpontzeko, logika bi metodo berritan isolatu dut. Alde batetik, erabiltzailearen saldoaren egiaztapena diruNahikoaDu metodora atera dut. Bestetik, saskiko elementu bakoitzaren erosketa prozesamendua erosketaBakarra metodoan banatu dut. Era honetan, jatorrizko metodoaren adar kopurua txikitu dut, irakurgarritasuna eta mantengarritasuna nabarmen hobetuz.

4. Egilea: Danel Larreategui